Smells and Refactoring

Improving Design and Code Safely

FinTech 512

Duke

Bridge from Previous Lectures

Where We Are in the Course

- **OO Design:** Make design decisions explicit
- **Design Principles:** Evaluate those decisions
- **Refactoring:** Improve decisions *without changing behavior*

Key shift

- Design does not stop when code is written
- Understanding evolves → code must evolve with it

What is Refactoring?

- Changing the **structure** of code
- Without changing its **observable behavior**
- Goal: make code easier to understand and change

We rarely understand the problem fully at the start

- Requirements change
- Domain knowledge deepens
- Early designs accumulate **friction**

Refactoring is how we *pay down misunderstanding*, safely.

Duke

What Are Code Smells?

- Signs that code may have **design problems**
- Not bugs
- Not errors
- Indicators of **misplaced responsibilities or unclear intent**

Smells tell us *where* refactoring might help.

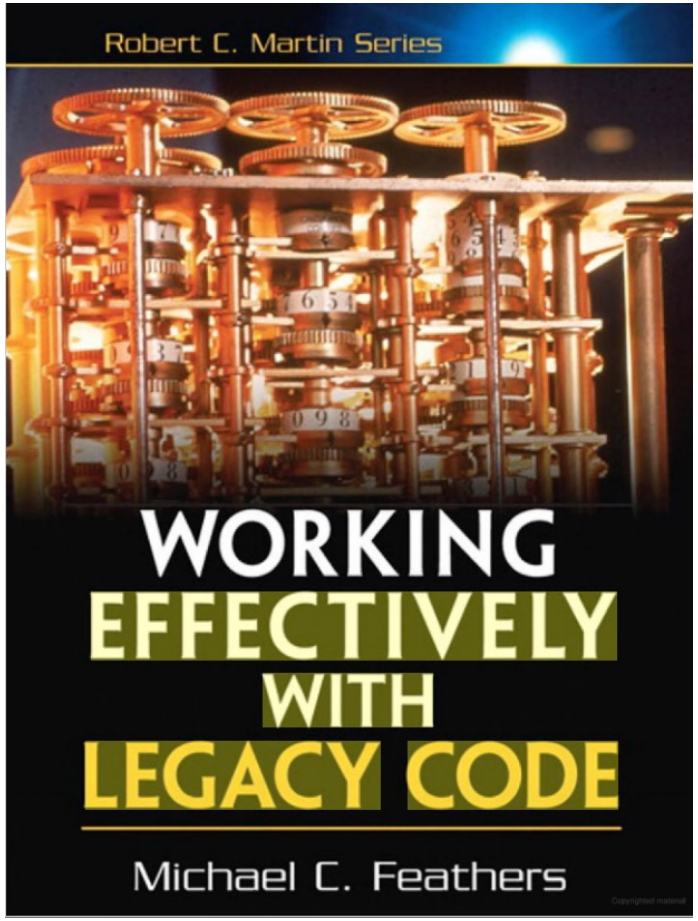
Code Reflects Knowledge

Code encodes:

- Business rules
- Assumptions
- Terminology
- Decisions we believed were correct at the time

Problem: When knowledge changes but code does not, the code becomes misleading

‘Working Effectively With Legacy Code’



“To me, *legacy code* is simply code without tests.”

- Michael Feathers

Duke

Why Legacy Code is Hard to Change

The Real Problem with Legacy Code

- We don’t know what behavior must be preserved
- We don’t know if a change broke something
- Fear replaces confidence

Change involves risk

To mitigate this risk, we ask:

1. What changes do we have to make?
2. How will we know that we’ve done them correctly?
3. How will we know that we haven’t broken anything?

Duke

Approaches to Change

Edit And Pray

- Study, plan, understand the code to be changed, carefully make the changes
- Manually test the system to see if behavior is correct
- used to be the industry standard, still common

Cover and Modify

- Create, run, tests to check behavior
- Study, plan, understand the code to be changed, carefully make the changes
- Regression test

Regression Testing

- Running tests to ensure **existing behavior has not changed**
- After making a change to the code
- A *regression* is behavior that used to work, but no longer does

What regression testing is *not*

- Not a different kind of test
- Not only about finding new bugs
- Not just for refactoring

Did we accidentally break something?

Duke

Regression Testing

A good regression test suite makes you **unafraid** of changing code

What do I mean and why?

Regression Testing

A good regression test suite makes you **unafraid** of changing code

Refactoring Requires Safety

- Change is risky
  - Refactoring changes structure
  - Structure changes can introduce defects
- Tests provide confidence
- Regression tests protect behavior

Characterization Tests

- Tests that capture **current behavior**
- Even if that behavior is ugly, surprising, or wrong
- The goal is to **preserve behavior**, not validate correctness

When you use them

- Code has no tests
- Behavior is unclear
- Refactoring is risky

Sensing & Subsetting

Sensing

- Add tests to **observe outputs**
- Log results, return values, side effects
- Treat the code as a **black box**

Subsetting

- Isolate a smaller piece of behavior
- Reduce inputs and dependencies
- Gradually shrink the “unknown” surface

Code Smells

- Signs that code may have **deeper design problems**
- Not bugs or syntax errors
- Indicators that **design decisions are no longer clear**

Why smells exist

- Understanding evolves
- Requirements change
- Code was written with **older assumptions**
- Structure no longer reflects intent

Duke

Taxonomy of Code Smells

Category	Examples	Description
Bloaters	Long Method, Large Class, Long Parameter List, Primitive Obsession, Data Clumps	Represent something that has grown so large that it cannot be effectively handled.
Object-Orientation Abusers	Switch Statements, Temporary Field, Refused Bequest, Alternative Classes w/ Different Interfaces	Represent cases where the solution does not fully exploit the possibilities of object-oriented design.
Change Preventers	Divergent Change, Shotgun Surgery, Parallel Inheritance Hierarchies	Hinder changing or further developing the software
Dispensables	Duplicate Code, Dead Code, Lazy Class, Data Class, Speculative Generality	Represent something unnecessary that should be removed from the source code.
Couplers	Feature Envy, Inappropriate Intimacy, Message Chains, Middle Man	Excessive or inappropriate coupling between classes

Taxonomy adopted from Mäntylä & Lassenius (Empirical Software Engineering, 2006).
https://mmantyla.github.io/BadCodeSmellsTaxonomy

Smell: Mysterious Name

What this smell looks like

- Names that are vague, misleading, or overly generic
- Names that require reading the implementation to understand
- Abbreviations or overloaded terms with unclear meaning

Why this matters

- Names encode **knowledge and intent**
- Poor names hide design decisions
- Confusing names make change and review harder

How to respond

- Rename to reflect **responsibility**, not implementation
- Use domain language, not technical shortcuts
- Treat renaming as the **safest form of refactoring**

```
def process(x):
    if x > 10000:
        return x * 0.001
    return x * 0.002
```

Duke

Now, how about …

```python
def calculate_transaction_fee(trade_amount_usd):
    LARGE_TRADE_THRESHOLD = 10000
    LARGE_TRADE_FEE_RATE = 0.001
    STANDARD_FEE_RATE = 0.002

    if trade_amount_usd > LARGE_TRADE_THRESHOLD:
        return trade_amount_usd * LARGE_TRADE_FEE_RATE

    return trade_amount_usd * STANDARD_FEE_RATE
```

Duke

Smell: Duplicated Code

What this smell looks like

- Same logic appears multiple places
- Small variations of the same calculation or rule
- Copy–paste with minor edits

Why this matters

- Duplication hides **shared intent**
- A single change requires **multiple edits**
- Inconsistent fixes create bugs

How to respond

- Identify what the code has in common
- Give that shared behavior a **name**
- Move it to one place

Duke

Duplicative Code Example

```python
class PaymentProcessor:
    def process_domestic(self, amount, balance):
        # Calculation + Validation Logic
        fee = amount * 0.01 # 1% domestic fee
        total = amount + fee
        if balance < total:
            raise ValueError("Insufficient funds")
        print(f"Domestic payment of ${amount} successful.")


    def process_international(self, amount, balance):
        # Calc. + Validation Logic (Identical structure, different value)
        fee = amount * 0.05 # 5% international fee
        total = amount + fee
        if balance < total:
            raise ValueError("Insufficient funds")
        print(f"International payment of ${amount} successful.")
```

Duke

Refactoring — Extract Method

When you use Extract Method

- You see duplicated or near-duplicated code
- A block of code has a clear purpose
- You want to **name the intent**

What it achieves

- Removes duplication
- Makes shared behavior explicit
- Improves readability and change safety

Duke

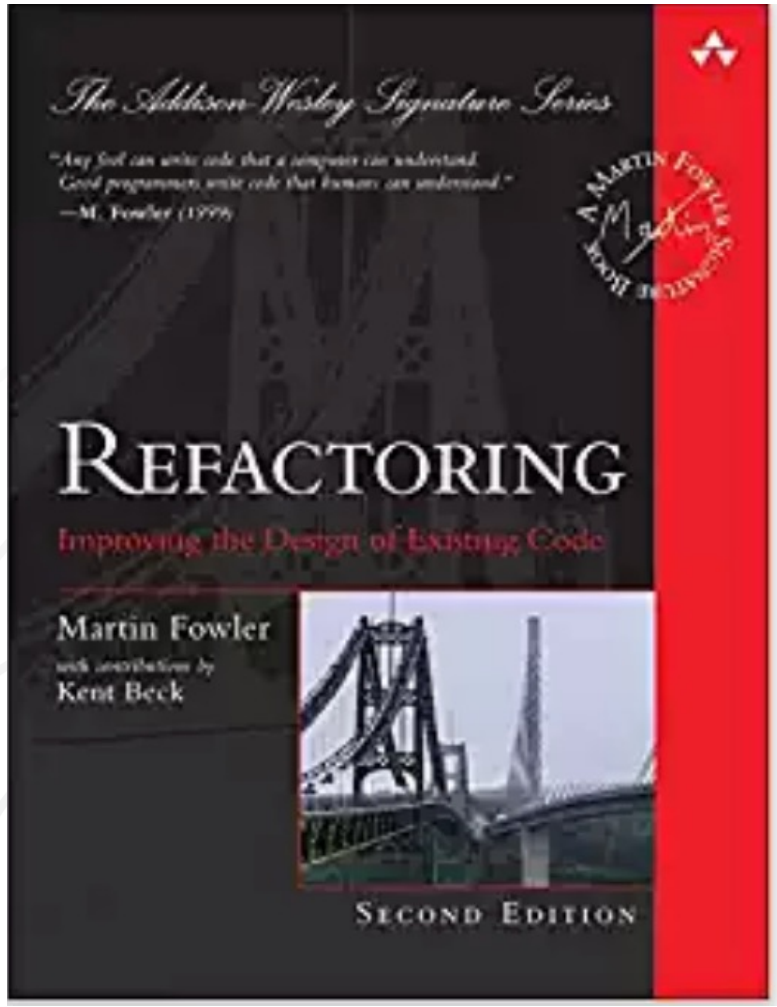
After: Duplication Removed

```python
class PaymentProcessor:
    def _process_payment(self, amount, balance, fee_rate, label):
        fee = amount * fee_rate
        total = amount + fee
        if balance < total:
            raise ValueError("Insufficient funds")
        print(f"{label} payment of ${amount} successful.")


    def process_domestic(self, amount, balance):
        self._process_payment(amount, balance, 0.01, "Domestic")


    def process_international(self, amount, balance):
       self._process_payment(amount, balance, 0.05, "International")
```

Duke

Refactoring as a Discipline

- **Martin Fowler — *Refactoring***
- Refactoring is a **named, systematic practice**
- Each refactoring:
  - is small
  - preserves behavior
  - improves structure
- Refactorings can be **combined safely over time**



Refactoring is how professionals improve design *without breaking working code*.

Taxonomy of Refactorings

Category	Examples	Description
Composing Methods	Extract Method, Inline Method, Replace Temp with Query, Substitute Algorithm.	Focuses on streamlining methods to remove duplication and clarify logic.
Moving Features Between Objects	Move Method, Move Field, Extract Class, Hide Delegate, Remove Middle Man.	Deals with moving functionality between classes to ensure responsibilities are in the right place.
Organizing Data	Replace Magic Number with Symbolic Constant, Replace Type Code with Class, Encapsulate Field.	Aims to make working with data easier by replacing primitives with objects or changing how associations are handled.
Simplifying Conditional Expressions	Decompose Conditional, Replace Nested Conditional with Guard Clauses, Replace Conditional with Polymorphism.	Targets complex logic gates and if/else chains to make the code's path easier to follow.
Simplifying Method Calls	Rename Method, Add Parameter, Separate Query from Modifier, Replace Parameter with Explicit Methods.	Makes interfaces and APIs easier to understand by adjusting how methods are named or what parameters they require.
Dealing with Generalization	Pull Up Method, Push Down Field, Extract Subclass, Extract Interface, Form Template Method.	Specifically manages the inheritance hierarchy, moving methods and fields up or down the tree.

Smell: Primitive Obsession

What this smell looks like

- Domain concepts represented as primitives (str, int, float)
- Repeated checks/formatting/parsing scattered through the code
- “Magic” values with special meaning (rates, currency, thresholds)

Why this matters

- Primitives hide domain rules and constraints
- Validation and formatting get duplicated
- Code becomes harder to change safely

How to respond

- Introduce a small domain type (e.g., Money, Percent, AccountBalance)
- Move validation and formatting into that type
- Replace magic values with named concepts

```
def can_execute_trade(amount, currency,
                      balance):
  if currency != "USD":
      raise ValueError("Unsupported currency")

  fee = amount * 0.02
  return balance >= amount + fee
```

Duke

Refactoring — Replace Primitive with Object

When you use Replace Primitive with Object

- A primitive (str, int, float) is carrying **domain meaning**
- You keep repeating validation, formatting, or conversion logic
- A concept has rules, but there's **no place for them to live**

What it achieves

- Introduces a small domain type (e.g., Money, Percent, Currency)
- Moves rules and invariants into that type
- Replaces scattered checks with a **single, named concept**

```python
class Money:
    def __init__(self, amount, currency):
        if currency != "USD":
            raise ValueError("Unsupported currency")
        self.amount = amount
        self.currency = currency

    def __add__(self, other):
        if self.currency != other.currency:
            raise ValueError("Cannot add money in different currencies")

        return Money(self.amount + other.amount, self.currency)

    def __ge__(self, other):
        return self.amount >= other.amount

class FeePolicy:
    def fee_for(self, trade_amount: Money) -> Money:
        return Money(trade_amount.amount * 0.02, trade_amount.currency)

    def can_execute_trade(trade_amount: Money, balance: Money, fee_policy: FeePolicy):
        fee = fee_policy.fee_for(trade_amount)
        return balance >= trade_amount + fee
```

Duke

Refactoring:
Replace Magic Value with Named Constant

When you use it

- A literal value has **implicit domain meaning**
- The same value appears in multiple places
- Changing the value would require searching code

What it does

- Gives the value a **name**
- Makes intent explicit
- Centralizes future change

Duke

Refactoring:
Replace Magic Value with Named Constant

```
STANDARD_FEE_RATE = 0.02


class FeePolicy:
    def fee_for(self, trade_amount: Money) -> Money:
        fee_amount = trade_amount.amount * STANDARD_FEE_RATE
        return Money(fee_amount, trade_amount.currency)
```

Duke

Refactoring Process

- Understand behavior (read the code)
- **Add/strengthen regression tests**
- Identify next smell (where change hurts)
- Apply *smallest* refactoring
- **Run tests**
- Repeat

- What smells bad?
- How would you refactor?

```
def buyStock(sym, qty):
    m = None
    for m2 in allStockMarkets:
        if m2.tradesSymbol(sym):
            m = m2
    if m == None: # invalid symbol
        return  -1
    if not m.isOpenNow():
        m.deferOrder(sym, qty)
        return 1 # indicate order waiting for market to open
    if (accountBalance < m.getPrice(sym) * qty):
        return 2 # indicate insufficient funds
    accountBalance -= m.getPrice(sym) * qty
    m.purchase(sym,qty)
    return 0 # indicate success
```

buyStock("AAPL",200)

Duke

```
def buyStock(sym, qty):
    m = None
    for m2 in allStockMarkets:
        if m2.tradesSymbol(sym):
            m = m2
    if m == None: # invalid symbol
        return  -1
    if not m.isOpenNow():
        m.deferOrder(sym, qty)
        return 1 # indicate order waiting for market to open
    if (accountBalance < m.getPrice(sym) * qty):
        return 2 # indicate insufficient funds
    accountBalance -= m.getPrice(sym) * qty
    m.purchase(sym,qty)
    return 0 # indicate success
```

This really should be its own method.. Probably **duplicated** elsewhere in code

Duke

Refactoring: Extract Method

```
def findMarketFor(sym):
    m = None
    for m2 in allStockMarkets:
        if m2.tradesSymbol(sym):
            m = m2
    return m
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        return  -1
    if not m.isOpenNow():
        m.deferOrder(sym, qty)
        return 1 # indicate order waiting for market to open
    if (accountBalance < m.getPrice(sym) * qty):
        return 2 # indicate insufficient funds
    accountBalance -= m.getPrice(sym) * qty
    m.purchase(sym,qty)
    return 0 # indicate success
```

Duke

```
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        return -1
    if not m.isOpenNow():
        m.deferOrder(sym, qty)
        return 1 # indicate order waiting for market to open
    if (accountBalance < m.getPrice(sym) * qty):
        return 2 # indicate insufficient funds
    accountBalance -= m.getPrice(sym) * qty
    m.purchase(sym,qty)
    return 0 # indicate success
```

Magic Literals
Primitive Obsession

Duke

```
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        return  "Invalid Symbol"
    if not m.isOpenNow():
        m.deferOrder(sym, qty)
        return "Deferred" # indicate order waiting for market
    if (accountBalance < m.getPrice(sym) * qty):
        return "Insufficient Funds" # indicate insufficient fu
    accountBalance -= m.getPrice(sym) * qty
    m.purchase(sym,qty)
    return "Ok" # indicate success
```

Is this any better?

Duke

```
from enum import Enum
OrderResult = Enum('OrderStatus',['InvalidSymbol','Deferred',
'InsufficientFunds', 'Ok'])
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        return  OrderResult.InvalidSymbol
    if not m.isOpenNow():
        m.deferOrder(sym, qty)
        return OrderResult.Deferred # indicate order waiting fo
    if (accountBalance < m.getPrice(sym) * qty):
        return OrderResult.InsufficientFunds # indicate insuffi
    accountBalance -= m.getPrice(sym) * qty
    m.purchase(sym,qty)
    return OrderResult.Ok # indicate success
```

What about an enum?

Duke

```
class OrderResult:
    sym
    qty
    status # Enum('OrderStatus',['Deferred', 'Ok’])
    …
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        raise OrderExceptionInvalidSymbol(sym)
    if not m.isOpenNow():
        return m.deferOrder(sym, qty)
    if (accountBalance < m.getPrice(sym) * qty):
        raise OrderExceptionInsufficientFunds(sym,\
            accountBalance,m.getPrice(sym) * qty)
    accountBalance -= m.getPrice(sym) * qty
    return m.purchase(sym,qty)
```

What about an object?
And two of these are error cases: we have exceptions, use them?

Duke

```
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        raise OrderExceptionInvalidSymbol(sym)
    if not m.isOpenNow():
        return m.deferOrder(sym, qty)
    if (accountBalance < m.getPrice(sym) * qty):
        raise OrderExceptionInsufficientFunds(sym,\
            accountBalance,m.getPrice(sym) * qty)
    accountBalance -= m.getPrice(sym) * qty
    return m.purchase(sym,qty)
```

Duplication of code!
(But its only one expression??)

Duke

```
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        raise OrderExceptionInvalidSymbol(sym)
    if not m.isOpenNow():
        return m.deferOrder(sym, qty)
    if (accountBalance < (m.getPrice(sym) * qty) + commission)
        raise OrderExceptionInsufficientFunds(sym,\
            accountBalance,(m.getPrice(sym) * qty) + commissio
    accountBalance -= (m.getPrice(sym) * qty) + commission
    return m.purchase(sym,qty)
```

Suppose we change this to add a commission.
Have to do in all three places!

Duke

```
def buyStock(sym, qty):
    m = findMarketFor(sym)
    if m == None: # invalid symbol
        raise OrderExceptionInvalidSymbol(sym)
    if not m.isOpenNow():
        return m.deferOrder(sym, qty)
    totalPrice = (m.getPrice(sym) * qty) + commission
    if (accountBalance < totalPrice):
        raise OrderExceptionInsufficientFunds(sym,\
              accountBalance,totalPrice)
    accountBalance -= totalPrice
    return m.purchase(sym,qty)
```

Extract Variable

```
def buyStock(sym, qty): # sym object rather than string
    m = sym.preferredMarket()
    if m == None: # invalid symbol
        raise OrderExceptionInvalidSymbol(sym)
    if not m.isOpenNow():
        return m.deferOrder(sym, qty)
    totalPrice = (m.getPrice(sym) * qty) + commission
    if (accountBalance < totalPrice):
        raise OrderExceptionInsufficientFunds(sym,\
              accountBalance,totalPrice)
    accountBalance -= totalPrice
    return m.purchase(sym,qty)

class Stock:
    sym = string()
    preferredMarket = findMarketFor(sym)
```

Why pass a string when we could pass a StockSymbol (that could e.g., know its own market)

Duke

```
def buyAsset(sym, qty):
    m = sym.preferredMarket()
    if m == None: # invalid symbol
        raise OrderExceptionInvalidSymbol(sym)
    if not m.isOpenNow():
        return m.deferOrder(sym, qty)
    totalPrice = (m.getPrice(sym) * qty) + commission
    if (accountBalance < totalPrice):
        raise OrderExceptionInsufficientFunds(sym,\
            accountBalance,totalPrice)
    accountBalance -= totalPrice
    return m.purchase(sym,qty)

class Asset:
    sym = string()
    preferredMarket = findMarketFor(sym)
```

Nothing specific to Stocks here: could be stocks, bonds, commodities

Duke

```
def buyAsset(sym, qty):
    m = m.getMarket()
    if not m.isOpenNow():
      return m.deferOrder(sym, qty)

    totalPrice = m.getPrice(sym) * qty
    if accountBalance < totalPrice:
      raise InsufficientFundsException(accountBalance,
                                       totalPrice))

    accountBalance -= totalPrice;
    return m.purchase(sym,qty);
```

Now we can re-use this code for any subclass of FinancialAsset:
Stocks, Bonds, Commodities, (other things we haven't thought of yet)

Duke

Code Smell Detective Exercise

```
def process_trade(t, u, a):
    if u.level == "premium":
        fee = a * 0.001
    else:
        fee = a * 0.002

    total = a + fee

    if u.balance < total:
        return -1

    u.balance -= total
    execute(t)
    return 0
```

- What smells can you find? (at least 3)
- Which refactorings would you apply?
- What would the improved code look like?

Duke

So What's Important?

Important

- Recognize common **code smells**
- Understand **why** a smell is a problem
- Know **which refactorings** address which smells
- Be able to **explain and apply** them to real code

Not the goal

- Memorizing long lists without understanding
- Applying refactorings mechanically
- Treating smells as rules instead of signals

Smells give us vocabulary.
Refactorings give us tools.
Judgment connects the two.

Duke

Correct In Design versus Fixing Later

Correct in Design vs. Fixing Later

- We rarely get the design “right” the first time
- Early code reflects **partial understanding**
- Later fixes reflect **better knowledge**

Key idea

- Good teams plan for **change**
- Refactoring is not failure — it’s **normal engineering**

Design quality is measured over time, not at first commit.

Duke

Fast versus Thorough

"Fast" often means:

- minimal upfront design
- learning by building

"Thorough" often means:

- more upfront thinking
- fewer early changes

Reality

- Speed without refactoring leads to decay
- Thoroughness without feedback leads to over-design

Professional development balances
speed now with **maintainability later**.

Duke

Big versus Small Changes

- Big rewrites are:
  - risky
  - hard to review
  - hard to test
- Small changes are:
  - easier to reason about
  - safer with tests
  - easier to roll back

Refactoring philosophy

- Prefer **many small, safe steps**
- Each step should preserve behavior

Dealing with Bad Design

When design feels wrong:

- Don’t rewrite immediately
- Don’t “clean everything up”
- Don’t add features on top of confusion

Instead:

- Identify **where change hurts**
- Name the **smell**
- Apply a **targeted refactoring**

Technical Debt

Technical debt is the **cost of deferred design improvement**

It accumulates when:

- smells are ignored
- refactoring is postponed
- shortcuts become permanent

Symptoms

- Changes take longer
- Bugs appear in unrelated areas
- Developers avoid touching code

Technical Debt… as originally defined

https://www.youtube.com/watch?v=pqeJFYwnkjE

Duke

How Do We Fix It (Pay Off Debt)?

- Pay off debt by fixing code: repair bad design
  - Refactor code: change design of code while maintaining behavior
  - Move from bad design to good design
- Note: do NOT try to do a ground-up rewrite
  - Basically never a good idea on any significant piece of software
  - Better to replace individual pieces and keep working system

Refactoring at Different Scales

Method level: *Extract Method, Inline Method*
→ Clarify intent, remove duplication

Class level: *Extract Class, Split Class*
→ Fix mixed responsibilities (SRP)

Module level: *Move Method, Move Field*
→ Put behavior with the data it belongs to

System level: *Extract Service*
→ Separate deployment, scaling, or ownership concerns

Using AI for Code Smells and Refactoring

How AI can help

- Identify potential **code smells**
- Suggest possible **refactorings**
- Surface **design alternatives**
- Ask questions you might not think to ask

What AI cannot do

- Understand full system context
- Know business intent or constraints
- Own design decisions
- Judge tradeoffs in your domain

Duke

Useful Prompt Patterns for Refactoring

Clarifying intent	What is this code trying to do at a business level?
Smell detection	Do you see any code smells here? Why?
Refactoring suggestions	What small refactorings could improve clarity without changing behavior?
Design critique	What responsibilities are mixed together in this code?
Change impact	If this logic changes, what other parts of the system might be affected?
Test gaps	What regression tests would make refactoring this safer?

Using Refactoring Guru
(and similar references)

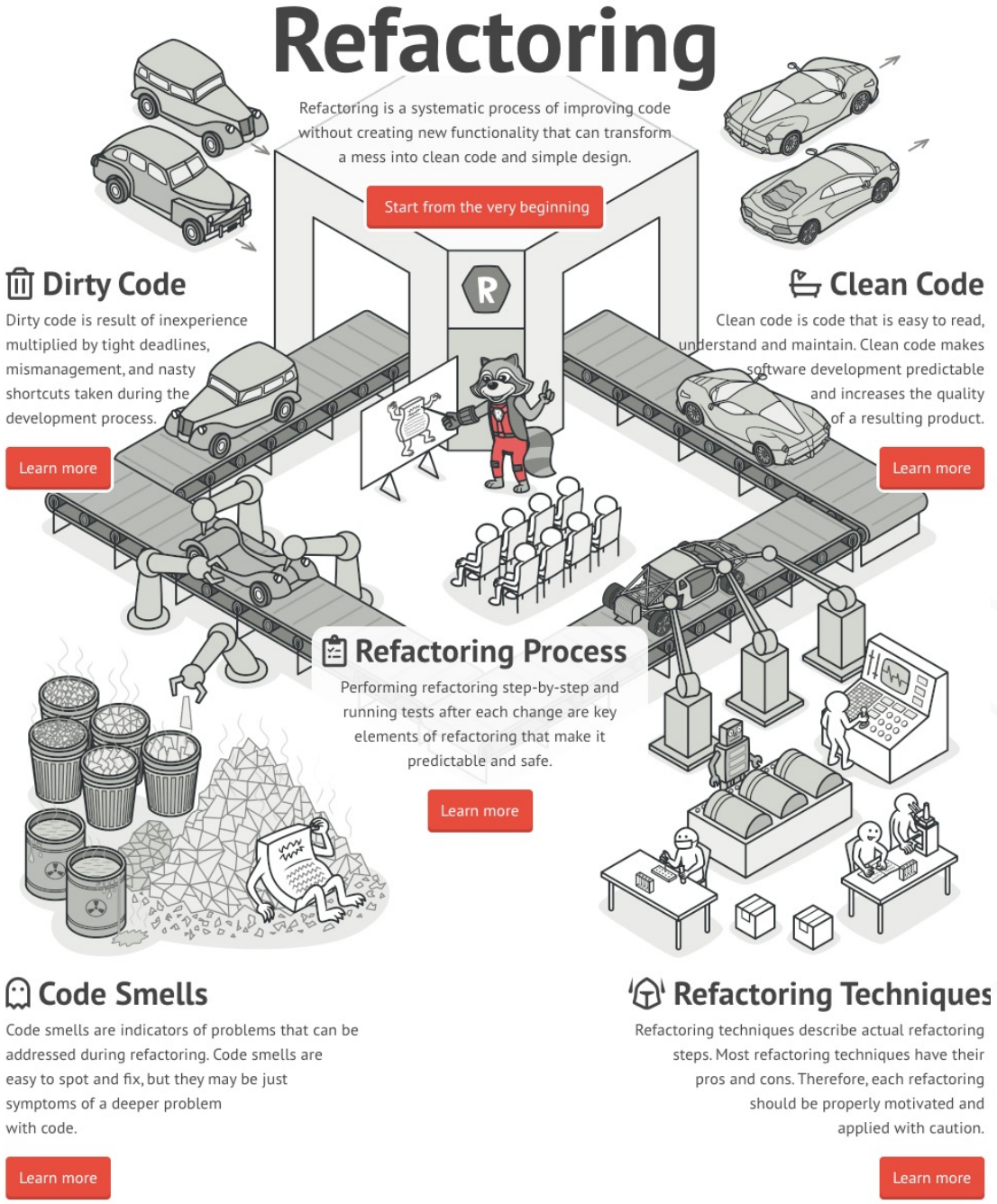
What these resources are good for

- Clear descriptions of common **code smells**
- Canonical **refactorings** and their intent
- Seeing **smell → refactoring** relationships

How to use them effectively

- Learn the **names and categories**
- Understand **why** a smell is a problem
- Know which refactorings typically address it
- Practice **recognizing them in real code**

- **What not to do**
- Don’t memorize lists without understanding
- Don’t treat refactorings as recipes
- Don’t apply refactorings without identifying the smell first



https://refactoring.guru/refactoring